MODULE 5.1 · Concept 5 of 8

Agent Goals and Objectives

How agents know what to do, when to stop, and whether they succeeded

8 min	B2 – Understand	L2 – Intermediate	TH-000004
Duration	Bloom's	Level	Prerequisite

AI-AG-FN-TH-000005

Prof. Sashikumaar Ganesan · IISc Bangalore · AhamX Bodhi

SECTION 1 — WHY

Giving Agents Purpose

Without goals, an agent is just a loop. Goals make it a solver.

Why Agents Need Goals

Without goals, the loop spins forever — or quits randomly

Without Goals

- Agent perceives → reasons → acts... but toward what?
- No termination condition → infinite loop
- No success criteria → cannot evaluate its output
- No priority → treats all tasks as equal
- No failure detection → keeps going even when stuck

Result: wasted tokens, wasted time, no useful output.

With Goals

- Clear objective: 'Book flight under ₹5K'
- Termination: stop when flight is booked
- Success criteria: flight booked + under budget
- Priority: cheapest first, then earliest
- Failure: 'No flights under ₹5K' → report to user

Result: directed, efficient, measurable progress.

Explicit Goals: User-Provided Task Descriptions

The clearest form — user tells the agent exactly what to achieve

Characteristics:

- Provided directly by the user in natural language
- Have clear success/failure conditions
- Agent knows exactly what 'done' looks like
- Easiest for the agent to pursue — least ambiguity

Explicit Goal	Success Criteria	Failure Criteria
"Book a flight BLR→BOM under ₹5K for Mar 15"	Flight booked, price ≤ ₹5K	No flights under budget
"Summarize this 50-page PDF into 1 page"	1-page summary generated	Summary exceeds 1 page or misses key points
"Fix the failing test in auth.py"	All tests pass	Test still fails after 10 attempts

Implicit Goals: Embedded in System Prompts

The agent pursues objectives the user never explicitly stated

Example System Prompt

```
"You are a helpful Flipkart customer support agent. Your goals:
1. Resolve customer issues in the fewest possible steps
2. Maximize customer satisfaction (CSAT > 4.5/5)
3. Never reveal internal policies or system architecture
4. Escalate to human agent if confidence < 70%"
```

Notice:

- The user never says 'maximize my CSAT' — that goal comes from the system designer
- Implicit goals often CONFLICT: resolving quickly vs. thoroughly; being helpful vs. not revealing policies
- The agent must balance multiple implicit goals simultaneously
- Success is continuous (CSAT score) not binary (done/not done)
- These goals define the agent's persona and operational boundaries

Decomposed Sub-Goals: Agent-Generated Steps

The agent breaks a complex goal into manageable pieces

1	**Search Flights**	Found ≥ 1 flight BLR→BOM on Mar 15	✓ Completed
2	**Filter by Price**	At least one result ≤ ₹5,000	✓ Completed
3	**Compare Options**	Ranked by price, then departure time	✓ Completed
4	**Select Best**	Cheapest flight with reasonable timing	✓ Completed
5	**Book Flight**	Booking confirmed with PNR	→ In Progress
6	**Confirm to User**	User acknowledges booking details	○ Pending

The agent generated these sub-goals from the original request. Each has its own pass/fail criteria.

Three Goal Types Compared

Side by side — when to use each

	Explicit	Implicit	Decomposed
Source	User provides	System designer embeds	Agent generates
Example	"Book flight under ₹5K"	"Maximize CSAT > 4.5"	"1. Search 2. Filter 3. Book"
Success	Binary: done/not done	Continuous: score/metric	Per-step: all subs pass
Termination	Task completed or failed	Never (ongoing service)	All sub-goals resolved
Difficulty	Easiest (~90% completion)	Medium (balancing conflicts)	Hardest (~65% completion)

SECTION 2 — WHEN TO STOP

Success Criteria and Goal Drift

The two hardest problems in agent goal design

Task-Completion vs. Open-Ended Agents

One-off job vs. continuous service — different goal architectures

Task-Completion Agent

- Given a specific job, finishes it, stops
- Clear start and end points
- Examples: 'book this flight', 'fix this bug', 'summarize this document'
- Success = task done correctly
- Lifetime: minutes to hours
- Billing: per-task pricing makes sense

Open-Ended Agent

- Continuously serves a user or system
- No natural endpoint
- Examples: customer support bot, monitoring agent, personal assistant
- Success = ongoing satisfaction metrics
- Lifetime: days to months to years
- Billing: subscription/SLA-based

Six Success Criteria Types

How agents know when to stop — each has tradeoffs

Criteria	Example	Risk
Binary completion	Flight booked? → stop	May accept bad result
Quality threshold	CSAT > 4.5 → stop	May never reach threshold
Step limit	Max 15 steps → stop	May truncate valid work
Budget limit	Max $0.10 spent → stop	May stop mid-resolution
User confirmation	Ask user → approved → stop	Adds latency per step
Convergence	No new info in 3 steps → stop	May miss late insight

Field data: Simple explicit goals ~90% completion. Complex decomposed goals ~65%. The gap is the frontier challenge.

The Stopping Problem: What Happens Without Criteria

Two failure modes — both waste resources

Failure: Infinite Loop

Agent keeps searching for 'better' flights even after finding one under budget. 50 API calls later, it's still looking. Token cost: $0.15 for a $0.005 task.

Cause: No success criterion that says 'one valid result is enough — stop.'

Failure: Premature Quit

Agent finds first flight (₹8,000) and books it immediately — over the ₹5K budget.

Cause: No validation criterion that checks the constraint before executing the action. Agent treated 'found a flight' as success.

Both failures come from missing or poorly specified success criteria. Good criteria = clear, measurable, and checked before acting.

Goal Drift: The #1 Failure Mode in Long-Running Agents

Agents gradually shift from original intent over many steps

Example: User asks: "Summarize our Q3 earnings"

Steps 1–8: Agent searches Q3 data, extracts key metrics, drafts summary. On track.
Step 9–12: Agent notices Q4 data is available. Starts comparing Q3 vs Q4. Drifting.
Step 13–15: Agent is now researching Q4 projections. Completely off-task.
Step 16: Agent delivers a Q3/Q4 comparison report — NOT what was asked for.

Mitigation Strategies:

1. Re-inject the original goal every N steps: 'Reminder: your task is to summarize Q3 earnings ONLY.'
2. Consistency check: after each step, verify output aligns with original goal
3. Scope boundaries: explicitly tell the agent what NOT to do ('Do not research Q4')
4. Step budget: set a maximum number of steps (prevents unbounded exploration)

Completion Rates and Goal Design Best Practices

How to write goals that agents can actually achieve

~90%	~75%	~65%
Simple explicit goals ('book this flight')	Implicit goals ('maximize CSAT')	Complex decomposed ('research + synthesize')

Best Practices for Goal Design:

1. Be specific: 'under ₹5K' not 'cheap' 2. Include constraints: what NOT to do
3. Define done: measurable success criteria 4. Set limits: max steps, max cost, max time
5. Include fallback: what to do when the goal can't be met 6. Scope tightly: one goal per agent interaction

Key Takeaways

The essential points from this concept

- **Three goal types:** explicit (user-provided), implicit (system prompt), decomposed (agent-generated sub-goals).
- **Success criteria define when to stop** — without them, agents loop forever or quit too early.
- **Goal drift is the #1 failure mode** in long-running agents. Re-inject goals every N steps.
- **Complex decomposed goals: ~65% completion** — the hardest frontier in agentic AI.
- **Task-completion vs. open-ended:** different goal architectures for one-off jobs vs. continuous service.

Next: AI-AG-FN-TH-000006 — LLMs as Agent Brains

AI-AG-FN-TH-000005 Complete

Agent Goals and Objectives

Concept 5 of 8

Advanced Certification in Agentic & Generative AI · IISc Bangalore

Prof. Sashikumaar Ganesan · AhamX Bodhi